WinForms User Grid

Prueba técnica Wolters Kluwer

Mario Ventura Burgos

Abril 2026

Índice

1. Aplicación implementada....3
1.1 Descripción del proyecto....3
1.2 Funcionalidades....4
2. Buenas prácticas de programación....5
2.1 Arquitectura de 3 Capas para API (N-Tier)....5
2.2 Inyección de dependencias (DI) y SOLID....5
2.3 Middleware para centralizar gestión de errores....5
2.4 Programación Asíncrona (Async/Await)....5
2.5 UI Data-Driven y Extensibilidad del Modelo....6
2.6 Pruebas unitarias y testing....6

1. Aplicación implementada

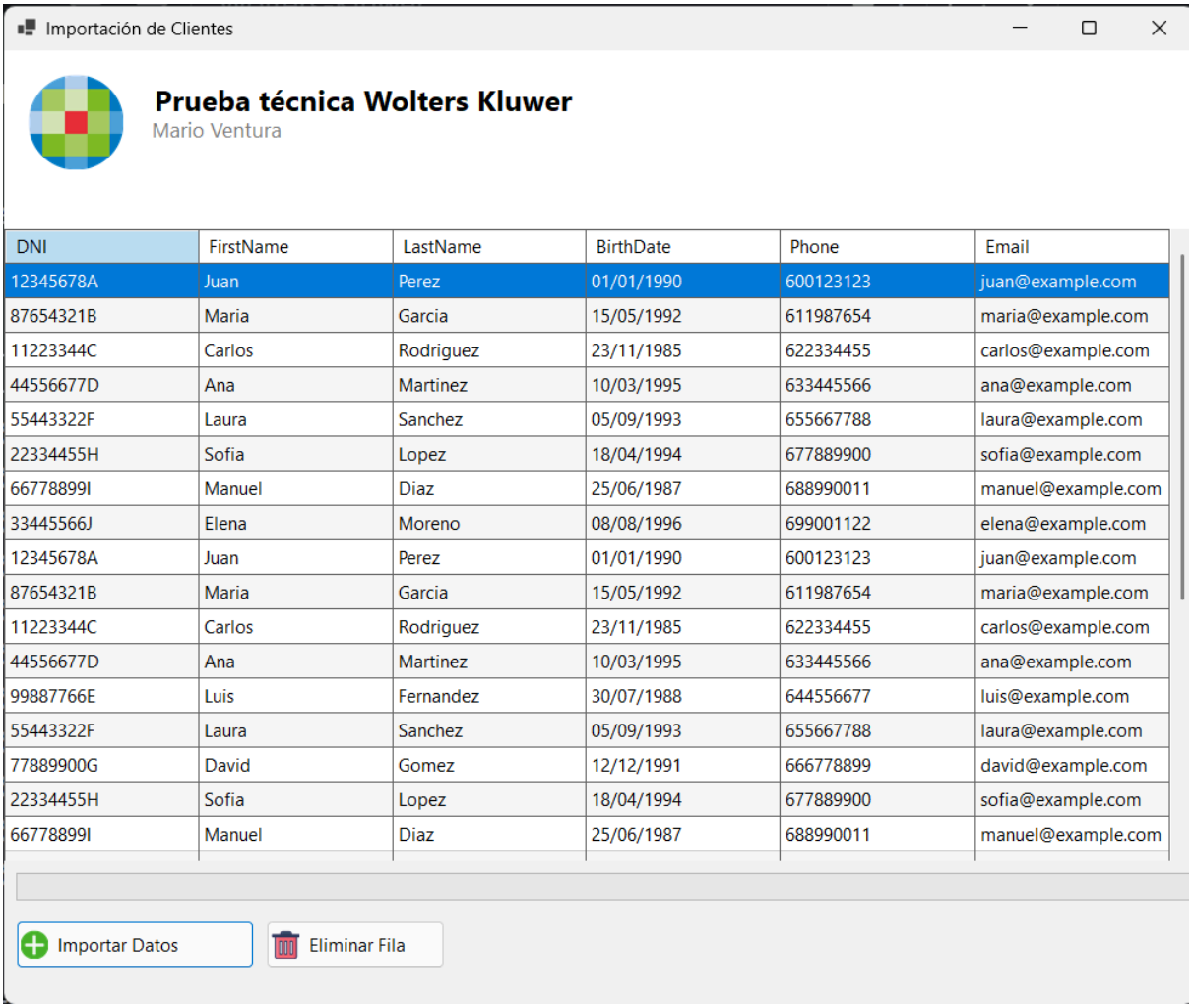
En este apartado se describe brevemente el proyecto y después se detallan sus funcionalidades

1.1 Descripción del proyecto

El proyecto consiste en una solución integral para la gestión y persistencia de datos de clientes. Se divide en dos componentes principales:

- **ClienteApi:** Una interfaz de programación (API REST) que actúa como el núcleo del sistema, gestionando la persistencia en formato JSON de forma segura y centralizada.
- **ClienteDesktop:** Una aplicación de escritorio desarrollada en WinForms que permite la visualización dinámica, importación y edición de clientes en tiempo real.

El objetivo principal ha sido transformar un prototipo funcional en una **herramienta empresarial escalable**, preparada para entornos de producción y cambios futuros en los requisitos de negocio.



DNI	FirstName	LastName	BirthDate	Phone	Email
12345678A	Juan	Perez	01/01/1990	600123123	juan@example.com
87654321B	Maria	Garcia	15/05/1992	611987654	maria@example.com
11223344C	Carlos	Rodriguez	23/11/1985	622334455	carlos@example.com
44556677D	Ana	Martinez	10/03/1995	633445566	ana@example.com
55443322F	Laura	Sanchez	05/09/1993	655667788	laura@example.com
22334455H	Sofia	Lopez	18/04/1994	677889900	sofia@example.com
66778899I	Manuel	Diaz	25/06/1987	688990011	manuel@example.com
33445566J	Elena	Moreno	08/08/1996	699001122	elena@example.com
12345678A	Juan	Perez	01/01/1990	600123123	juan@example.com
87654321B	Maria	Garcia	15/05/1992	611987654	maria@example.com
11223344C	Carlos	Rodriguez	23/11/1985	622334455	carlos@example.com
44556677D	Ana	Martinez	10/03/1995	633445566	ana@example.com
99887766E	Luis	Fernandez	30/07/1988	644556677	luis@example.com
55443322F	Laura	Sanchez	05/09/1993	655667788	laura@example.com
77889900G	David	Gomez	12/12/1991	666778899	david@example.com
22334455H	Sofia	Lopez	18/04/1994	677889900	sofia@example.com
66778899I	Manuel	Diaz	25/06/1987	688990011	manuel@example.com

1.2 Funcionalidades

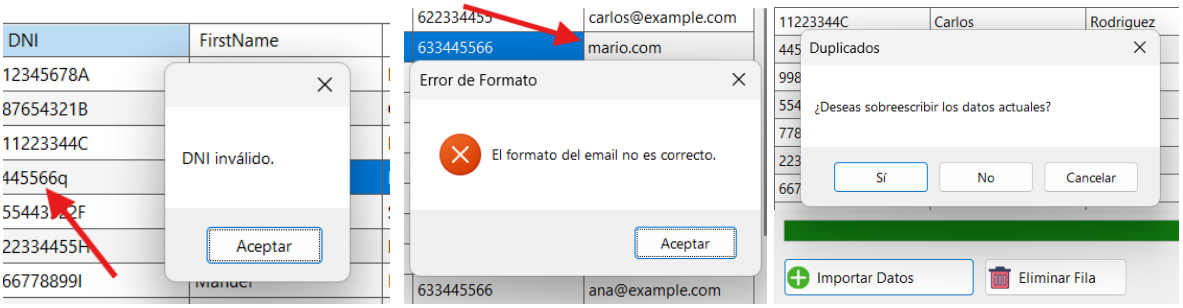
La aplicación implementada cuenta con las siguiente funcionalidades y características:

1. **Cargar datos en CSV:** El botón de importar datos, que aparece abajo a la izquierda junto con un icono verde con un "+", permite importar datos a la DataGridView implementada. Esta funcionalidad permite que los datos se importen desde un archivo en formato CSV.
2. **Cargar datos en JSON:** La importación de datos también permite que se haga desde ficheros en formato JSON.
3. **Sobreescritura de datos antiguos:** Si se intenta importar un usuario que ya existe (se comprueba mediante el DNI) la aplicación lo detecta e informa al usuario. Entones, el usuario puede decidir si sobreescribirlo, es decir, eliminar el antiguo y añadir el nuevo, o si prefiere tener el dato duplicado por alguna razón personal.
4. **Modificación sujeta a validación:** Todas las filas de la Grid permiten modificación in-line. Las columnas críticas, además, presentan una validación que, en caso de fallar, no permite que el dato sea modificado y lo devuelve al estado anterior.

   Estas validaciones se hacen desde ficheros específicos para validación de datos, y se llevan a cabo mediante expresiones regulares creadas con ayudas de herramientas de IA Generativa. Las validaciones hacen las siguientes comprobaciones:

   - DNI: Debe seguir el formato de 8 números y una letra.
   - Email: Debe seguir un formato que cumpla con el típico texto@dominio.extensión
   - Teléfono: Deben ser exactamente 9 dígitos.

5. **Guardado automático a fichero JSON:** Se produce una operación de guardado automática cada vez que una columna se modifica, añade, etc. Cada vez que esto sucede, un fichero llamado clients_store.json es modificado.
6. **Recordar datos de sesión anterior:** El fichero mencionado previamente sirve como store de datos. Si existe, la app lo carga al inicio porque entiende que son los datos que habían la última vez que se usó la app. Si no existe, la app crea el fichero la primera vez que se producde una importación de datos o creación de fila.
7. **Añadir nueva fila:** Se pueden añadir nuevas filas de forma manual a la grid existente simplemente rellenando los valores de cada columna. Obviamente, los datos que se introduzcan también serán validados.
8. **Eliminar fila:** Si se selecciona una fila y se presiona el botón de eliminar fila, la fila desaparecerá de la grid y, por tanto, también del fichero store de datos. Si se selecciona una celda en concreto y se pulsa el botón, se elimina la fila a la que pertenece dicha celda, de modo que no es necesario seleccionar la fila completa.



Además, cuenta con una **barra de progreso** que muestra el progreso de la importación/carga de datos mencionada en las funcionalidades 1 y 2.

2. Buenas prácticas de programación

Durante el desarrollo, se ha aplicado una reingeniería profunda basada en estándares de la industria:

2.1 Arquitectura de 3 Capas para API (N-Tier)

El mayor riesgo de una aplicación pequeña es el "Código Espagueti" (donde la lógica de guardado está mezclada con la respuesta HTTP). Se ha implementado una **Arquitectura de Responsabilidad Segregada**:

- **Capa de Presentación (Controllers):** Actúa como un *Gatekeeper*. Solo se encarga de recibir el JSON, validar que el formato sea correcto y devolver el código de estado HTTP adecuado (`200 OK`, `400 BadRequest`, `500 Error`). No sabe *cómo* se guarda un cliente, simplemente es una lista de endpoints.

- **Capa de Negocio (Managers):** Es el cerebro. Aquí reside el "saber hacer" de la empresa. Verifica si un DNI ya existe o si los datos cumplen las reglas. Al estar aislada, esta lógica se puede reutilizar en una App Web, Móvil o Desktop sin cambiar una sola línea.

- **Capa de Persistencia (Services):** Es el brazo ejecutor. Se encarga de la comunicación con el "mundo exterior" (en este caso, archivos JSON). Si mañana la empresa decide usar SQL Server o Azure CosmosDB, solo se sustituye esta capa.

2.2 Inyección de dependencias (DI) y SOLID

Para cumplir con el principio de Inversión de Dependencias (D en SOLID), las clases no crean sus propias herramientas, sino que las reciben por el constructor mediante la implementación de interfaces. El ClientManager, por ejemplo, no depende de la clase ClientService, sino de la interfaz IClientService. Esto permite el Mocking en las pruebas unitarias.

Durante el test, le pasamos al Manager un servicio falso que no toca el disco, garantizando que los tests sean ultrarrápidos y no dejen basura en el ordenador del desarrollador.

2.3 Middleware para centralizar gestión de errores

En lugar de llenar el código de bloques try-catch repetitivos, se ha implementado un Middleware de Excepciones Global. Cada petición atraviesa (en caso de usar API con BD) una serie de capas. Actúa como capa de seguridad en el exterior que captura cualquier fallo en cualquier nivel inferior.

Esto evita la fuga de información sensible. Si ocurre un error de base de datos, el Middleware lo registra en el log interno (con todos los detalles técnicos) pero le devuelve al usuario un mensaje genérico y elegante, protegiendo la infraestructura de posibles ataques.

2.4 Programación Asíncrona (Async/Await)

Se ha aplicado el patrón TAP (Task-based Asynchronous Pattern). En una API moderna, esto es vital debido a las razones siguientes:

- **No Bloqueo de Hilos (Thread Pool):** Cuando la API lee el disco, el hilo que atiende la petición se libera para atender a otros usuarios mientras el sistema de entrada/salida (I/O) trabaja. Esto permite que la API soporte miles de usuarios concurrentes con muy pocos recursos.
- **Experiencia en Desktop:** En la App WinForms, el uso de async/await evita que la ventana se quede "congelada" o en blanco mientras se importan miles de registros, mejorando drásticamente la percepción de calidad del usuario.

2.5 UI Data-Driven y Extensibilidad del Modelo

Uno de los requisitos implícitos de una aplicación escalable y profesional es la capacidad de evolución. Por ello, se ha implementado un método de creación de grid que crea dicha cuadrícula añadiendo columnas en función de cuantas columnas tienen los datos. El cliente Desktop utiliza ***AutoGenerateColumns***. Esta técnica inspecciona los metadatos de la clase Client en tiempo de ejecución.

Por tanto, si el negocio decide mañana que los clientes deben tener un campo "Categoría", el desarrollador solo lo añade a la clase Client en la API. Automáticamente, el JSON lo guardará y el Grid del Desktop creará la nueva columna sin que nadie tenga que rediseñar el formulario manualmente.

2.6 Pruebas unitarias y testing

Se ha establecido una base de TDD para proteger el núcleo del negocio:

- **Pruebas de Regresión:** Cada vez que se añade una función, se ejecutan los tests automáticos para asegurar que lo nuevo no ha estropeado lo que ya funcionaba.
- **Validación de Casos de Borde (Edge Cases):** Los tests verifican comportamientos límite, como listas vacías, DNIs duplicados o fallos en el sistema de archivos, asegurando que la aplicación cubra todos los casos posibles.

```
PROBLEMAS   SALIDA   CONSOLA DE DEPURACIÓN   TERMINAL   ···   powershell - ClienteApi.Tests

PS C:\Users\User\Desktop\Wolters-Kluwer\PruebaTecnica\ClienteApi.Tests> dotnet test
Restauración completada (0,9s)
  ClienteApi net10.0 correcto con 5 advertencias (1,3s) → C:\Users\User\Desktop\Wolters-Kluwer\PruebaTecnica
\ClienteApi\bin\Debug\net10.0\ClienteApi.dll
```

...

```
  ClienteApi.Tests net10.0 realizado correctamente (0,3s) → bin\Debug\net10.0\ClienteApi.Tests.dll
[xUnit.net 00:00:00.01] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.5)
[xUnit.net 00:00:00.53]   Discovering: ClienteApi.Tests
[xUnit.net 00:00:00.67]   Discovered:  ClienteApi.Tests
[xUnit.net 00:00:00.77]   Starting:    ClienteApi.Tests
[xUnit.net 00:00:01.06]   Finished:    ClienteApi.Tests
  ClienteApi.Tests net10.0 realizado correctamente prueba (2,5s)

Resumen de pruebas: total: 5; con errores: 0; correcto: 5; omitido: 0; duración: 2,5 s
Compilación correcto con 5 advertencias en 6,0s
PS C:\Users\User\Desktop\Wolters-Kluwer\PruebaTecnica\ClienteApi.Tests>
```